AWS INFRASTRUCTURE ARCHITECTURE

AWS 인프라 구조와 설계 강점

출시 서비스를 받치는 구조, 그리고 그 구조를 고른 이유

기준	정본	배포 리전	운영 형태
develop 브랜치 7ce2a53 2026-10-30	`infra/terraform/stack` `deploy/eks/chart`	AWS 서울(ap-northeast-2)	EKS Auto Mode Terraform + Helm

SUMMARY

작은 팀이 운영할 수 있는 규모로, 큰 서비스와 같은 원칙을 지킵니다

01
3계층 네트워크 격리
DB·캐시 서브넷에는 인터넷으로 나가는 경로가 아예 없습니다.

02
중단을 줄이는 이중화와 복구
복제본 이중화, 계획된 교체 중 최소 개수 유지, 배포 실패 시 자동 롤백을 갖췄습니다.

03
키 없는 권한 모델
저장소와 파드 어디에도 오래 쓰는 AWS 키가 없습니다.

04
보호가 기본인 데이터
운영 DB는 암호화, 14일 시점 복구, 삭제 보호, 최종 스냅숏이 기본값입니다.

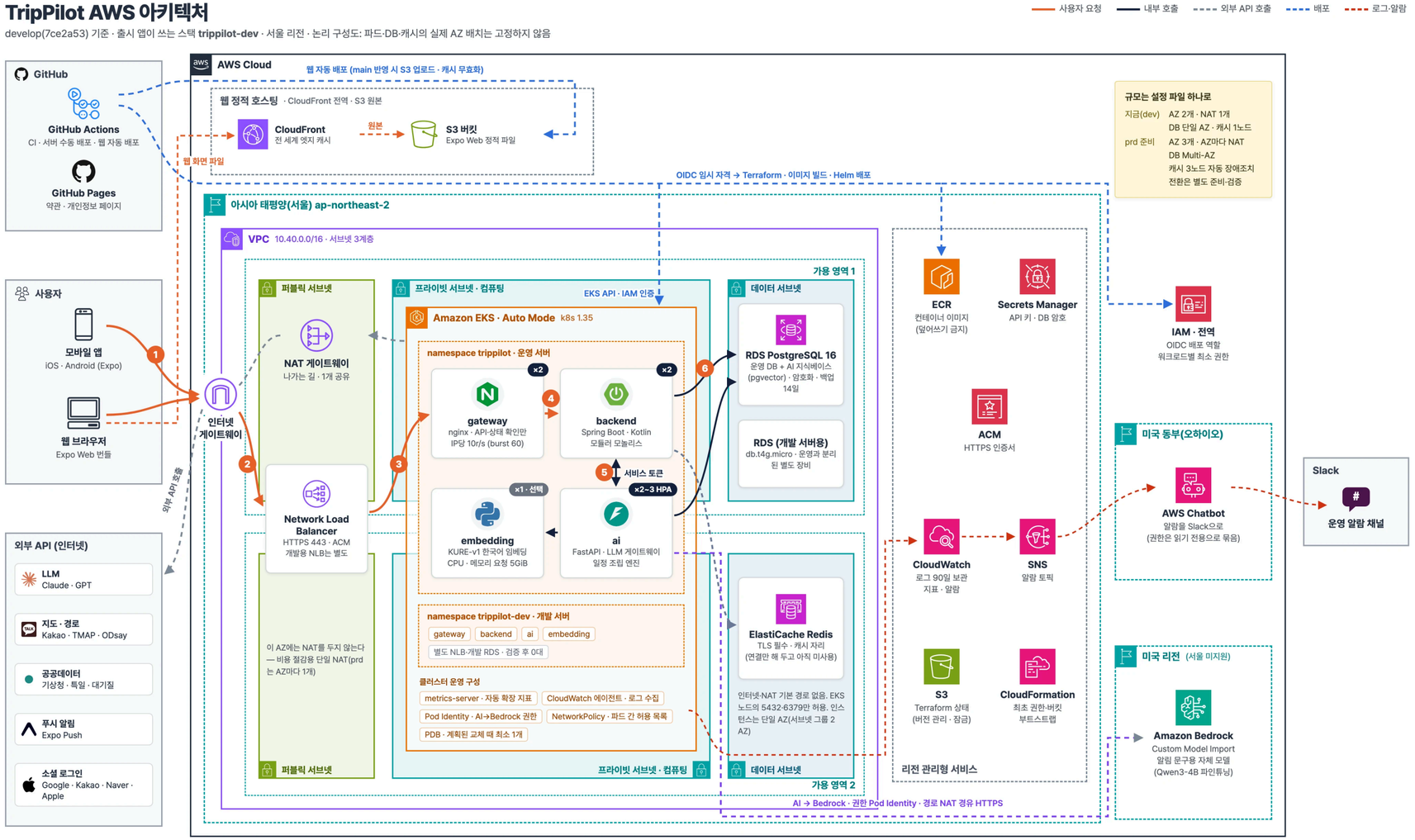
05
코드형 인프라와 배포 게이트
정기 릴리스는 개발 서버 배포에 성공한 코드만 운영에 나갑니다.

06
조용한 실패까지 잡는 관측
로그에서 이상 징후를 세어 Slack으로 바로 알립니다.

07 **비용에 비례하는 설계** 지금은 필요한 만큼만 쓰고, 운영 규모에 맞춘 3 AZ·Multi-AZ 설정을 미리 마련해 두었습니다.

ARCHITECTURE

전체 구조도



AWS 관리형 서비스

VPC · NLB · EKS · ECR · RDS · ElastiCache · Secrets Manager · IAM · ACM · CloudWatch · SNS · Chatbot · S3 · CloudFormation · Bedrock · CloudFront

외부 연동

LLM · 지도·경로 · 공공데이터 · Expo Push · 소셜 로그인

그림을 누르면 원본 크기로 볼 수 있습니다.

강점 01 · 다층 방어

여섯 겹의 통제로 데이터 계층을 지킵니다

TLS 종단
NLB · ACM
앱으로 들어오는 입구는 HTTPS 443 하나입니다.

API 게이트웨이
nginx
/api와 상태 확인만 통과시키고 나머지는 404로 막습니다. IP마다 초당 10건(순간 60건)으로 속도를 제한합니다.

파드 간 허용 목록
NetworkPolicy
backend는 gateway·ai와 배포용 Job에서만, ai는 backend와 점검 Job에서만 연결을 받습니다.

컨테이너 잠금
securityContext
root 실행 금지, 읽기 전용 파일시스템, 권한 상승 차단, capability 전부 제거.

보안 그룹
Security Group
DB 5432와 캐시 6379는 EKS 노드에서 오는 연결만 받습니다.

격리 서브넷
Data subnet
인터넷·NAT 기본 경로가 없습니다. DB는 SSL을, Redis는 TLS를 강제합니다.

한 겹이 뚫리더라도 다음 겹이 막도록 쌓았습니다. 특히 **데이터 계층의 라우팅 테이블에는 인터넷·NAT로 가는 기본 경로가 없습니다.** 보안 그룹을 잘못 열어도 바깥과 곧바로 이어지지 않습니다.

서비스 토큰과 JWT 서명키가 없으면 앱이 아예 기동하지 않도록 해, 인증 설정이 빠진 채 운영되는 일도 막습니다.

`network.tf · data.tf · gateway-config.yaml · network-policy.yaml · _helpers.tpl · backend.yaml`

강점 02 · 이중화와 복구

장애 상황마다 대응 장치를 하나씩 정해 두었습니다

상황	대응	설정
노드 교체·통합	복제본 2개와 PDB로 최소 1개를 남깁니다	`replicas: 2 · PDB minAvailable: 1`
노드 장애	EKS Auto Mode가 노드를 교체하고, 복제본은 AZ·노드별로 나눠 배치되도록 우선합니다	`topologySpread: zone · hostname`
트래픽 증가	AI 서버는 CPU 요청 대비 평균 사용률을 보고 2~3개 사이에서 늘고 줄어듭니다	`HPA 2~3 · 목표 70% · metrics-server`
배포 실패	API 파드는 새 파드가 준비된 뒤 옛 파드를 내립니다. Helm 배포가 실패하면 이전 릴리스로 되돌립니다	`maxUnavailable: 0 · helm --atomic`
긴 요청 중 종료	일정 생성처럼 오래 걸리는 요청이 마무리될 시간을 최대 660초 둡니다	`종료 유예 660초 · preStop drain`
LLM 공급자 장애	규칙 기반 폴백으로 응답을 이어 가고 생성 시간에 상한을 둡니다	`AI 폴백(INV-4) · 데드라인 15초/60초`

`values-dev.yaml · pdb.yaml · hpa.yaml · _helpers.tpl · .github/workflows/aws-deploy.yml`

강점 03 · 키 없는 권한 모델

오래 쓰는 AWS 키를 어디에도 두지 않습니다

CI/CD

GitHub OIDC

배포 실행마다 단기 자격을 받습니다. 지정된 GitHub 환경의 실행만 역할을 맡을 수 있습니다.

CLUSTER

EKS Access Entry

API 인증 모드만 씁니다. 클러스터를 만든 사람에게 자동으로 붙는 관리자 권한도 껐습니다.

WORKLOAD

Pod Identity

AI 파드에는 Bedrock 자체 모델 호출 권한 하나만 줍니다. 로그 수집기도 역할을 따로 씁니다.

CHATOPS

Chatbot 가드레일

Slack에서 할 수 있는 일의 상한을 CloudWatch 읽기 권한으로 묶었습니다.

SECRETS

비밀값은 Terraform state와 Helm values에 넣지 않고 Secrets Manager에서 Kubernetes Secret으로 동기화합니다. 비어 있는 값은 배포 도구가 생성합니다. RDS 관리자 암호는 앱 파드에 전달하지 않고 런타임과 마이그레이션 DB 계정을 나눴습니다.

`infra/bootstrap/template.json · eks.tf · registry-secrets.tf · alerts.tf · deploy/eks/runtime_secrets.py`

강점 04 · 데이터 보호

보호 장치는 '켜져 있는 것'이 기본값입니다

14일
운영 DB를 특정 시점으로
되돌릴 수 있는 기간

30일
삭제한 운영 비밀값을
되살릴 수 있는 기간

2중
운영 DB를 지워도 남는
최종 스냅숏과 자동 백업

0개
데이터 서브넷의
인터넷·NAT 기본 경로

빠뜨리면 보호가 켜지도록

삭제 보호·백업 보존을 묶은 스위치의 기본값은 '켜짐'입니다. 설정을 잊으면 안전한 쪽으로 틀리게 했습니다.

`infra/terraform/stack/data.tf · variables.tf(data_protection_enabled) · registry-secrets.tf`

지울 수 있어도 증거는 남도록

스위치를 꺼서 DB를 지울 수 있게 되더라도 최종 스냅숏과 자동 백업은 반드시 남깁니다. DB와 캐시는 저장할 때도, 주고받을 때도 암호화합니다.

강점 05 · 코드형 인프라와 배포 게이트

인프라는 코드로 관리하고 운영 반영에는 관문을 둡니다

GATE
브랜치·환경 검사
main→운영
develop→개발

→

GATE
개발 배포 확인
72시간 안 성공한 개발 배포와 코드가 같아야 통과

→

03
plan → apply
저장한 plan만 적용

→

04
이미지 발행
덮어쓰기 금지 태그
푸시 시 취약점 스캔

→

05
비밀값·DB 준비
멱등 실행
관리자 자격 즉시 삭제

→

06
Helm 배포
실패하면 자동 롤백

→

07
연결 확인
클러스터 안에서 API·AI 점검

핵심 인프라는 코드로

VPC부터 알람까지 Terraform, 권한·상태 버킷은 CloudFormation, 앱은 Helm 차트로 관리합니다. 웹 호스팅과 인증서만 최초 1회 따로 준비합니다.

상태 파일도 안전하게

환경마다 상태 버킷을 따로 두고 버전 관리·암호화·잠금을 켰습니다. destroy 워크플로는 만들지 않았습니다.

사람이 누르는 서버 배포

서버 인프라와 EKS 앱은 push로 바뀌지 않습니다. 누가 언제 실행했는지 기록이 남는 수동 실행으로만 반영합니다.

개발 배포 확인은 정기 릴리스 기준입니다. 긴급 수정(hotfix)은 Jira 키와 사유를 기록한 뒤 이 확인을 건너뛰고 사후에 검증합니다.

`.github/workflows/aws-deploy.yml · infra/scripts/deploy_gate.py · workflow_config.py · docs/guides/aws-eks-deployment.md`

강점 06 · 관측과 알람

장애 징후를 로그와 지표에서 잡아 팀에 바로 알립니다

알람	조건	잡아내는 것
게이트웨이 5xx	5분 10건 이상	서버 오류 급증
리프레시 토큰 재사용	1건이라도	계정 탈취 신호
Claude·GPT 호출 실패	30분 5건 이상	폴백에 가려진 AI 품질 저하
파드 재시작 · 노드 장애	재시작 증가 · NotReady 10분	OOM·기동 실패·노드 불량
DB CPU · 저장공간 · 연결	80%(15분) · 여유 1GiB · 150	용량 한계와 연결 누수

컨테이너 로그는 CloudWatch에 90일 보관하고 알람은 SNS → AWS Chatbot → Slack으로 전달합니다.

운영 사례 · 2026-10-28

외부 LLM 인증 오류로 하루 동안 호출 383건이 모두 실패했지만 폴백 덕분에 서비스는 정상 응답했고 알람도 울리지 않았습니다.

이 사례를 계기로 LLM 호출 실패를 직접 세는 알람을 추가했습니다. **장애에서 배운 것을 규칙으로 남기는 것**이 저희 운영 방식입니다.

`infra/terraform/stack/alerts.tf`

강점 07 · 비용에 비례하는 설계

지금은 필요한 만큼만 쓰고, 운영 규모 설정은 미리 마련했습니다

지금 아끼는 방법

- ✓ DB·캐시에 ARM 기반 Graviton(t4g) 인스턴스
- ✓ 개발 서버는 검증에 성공하면 0대로 축소
- ✓ 메모리 5GiB를 요청하는 임베딩은 필요할 때만 배포
- ✓ 알림 문구 모델은 Bedrock 서버리스, 유휴 시 추론 용량 0
- ✓ 실측으로 자원 요청값 조정(backend CPU 500m → 200m)
- ✓ 로그 보존 기한을 명시해 무기한 과금 방지

운영 규모로 올릴 때

항목	지금(DEV)	PRD 설정(준비 완료)
가용 영역	2	3
NAT 게이트웨이	1개 공유	AZ마다 1개
PostgreSQL	단일 AZ · t4g.small	Multi-AZ · t4g.medium
Redis	1노드	3노드 · 자동 장애조치

prd에서 3 AZ와 AZ별 NAT를 빼먹으면 Terraform 변수 검증에서 실패하도록 막아 두었습니다.

AI 워크로드

AI 기능도 인프라 원칙 안에서 돌아갑니다

분리 배포

ai · embedding

무거운 임베딩 모델(메모리 5GiB)을 따로 배포해 자원을 나누고, 임베딩이 멈추면 규칙 기반 추천으로 전환합니다.

자체 모델

Bedrock Custom Model Import

알림 문구는 직접 파인튜닝한 Qwen3-4B가 씁니다. 알림 문구 블라인드 비교에서 235B 교사 모델에 12:8로 앞섰습니다.

검증된 값만

조립 엔진

화면에 나가는 시간과 순서는 LLM이 아니라 최적화 엔진이 검증한 값입니다.

실패 대비

폴백 + 알람

Claude·GPT 호출이 실패하면 규칙 기반으로 응답하고, 실패 건수는 알람으로 따로 셉니다.

`ai/docs/ai-models-for-clients.md · deploy/eks/chart/templates/{ai,embedding}.yaml · alerts.tf`

운영 · 개발 분리

클러스터 하나로 비용을 아끼고 섞이면 안 되는 것은 나눴습니다



- ✓ 개발 DB는 별도 인스턴스라 데이터 변경·마이그레이션·부하가 운영 DB로 번지지 않습니다
- ✓ 도메인·인증서·시크릿·Bedrock 권한 연결을 따로 둡니다
- ✓ 앱 로그·재시작 알람은 운영 네임스페이스만 셉니다
- ✓ 개발 배포는 공유 인프라를 바꾸지 않고, 실제 푸시 발송도 강제로 끕니다
- ✓ 개발 서버는 검증에 성공하면 0대로 내립니다

```
variables.tf(dev_target_enabled) · values-dev-target.yaml · alerts.tf · workflow_config.py
```

검증 체계

인프라 코드도 테스트를 통과해야 AWS에 닿습니다

검증	지키는 것
terraform test	공인 IP 금지, DB 암호화·삭제 보호·백업 14일, 운영과 개발 자원 분리가 지켜지는지
infra/tests	워크플로 계약, 운영 배포 게이트, 환경 변수 파일 검증
infra/bootstrap/tests	IAM 역할·신뢰 정책 템플릿이 최소 권한을 지키는지
deploy/eks/chart/tests	Helm 차트 렌더링 결과(PDB·HPA·NetworkPolicy 등)
deploy/eks/tests	비밀값 동기화·DB 초기화·스모크 점검 도구

인프라를 건드리는 PR마다 **infra-validate** 워크플로가 Terraform 오프라인 검증, Helm 매니페스트 검증, 위 테스트를 자동으로 돌립니다.

CONCLUSION

정리

- ✓ **격리** — 데이터 계층은 인터넷 경로가 없는 격리 서브넷에 있습니다
- ✓ **회복** — 이중화·PDB·자동 롤백으로 노드 교체와 배포 때의 중단을 줄입니다
- ✓ **권한** — 장기 키 없이 OIDC와 Pod Identity로 필요한 권한만 줍니다
- ✓ **보호** — 운영 DB는 백업·삭제 보호·최종 스냅숏이 기본값입니다
- ✓ **재현** — 핵심 인프라가 코드이고 운영 반영에는 검증 게이트가 있습니다
- ✓ **관측** — 조용한 실패까지 Slack으로 알립니다
- ✓ **비용** — 지금 규모에 맞춰 쓰고, 운영 규모 설정은 미리 마련했습니다

다음 단계

prd 스택 분리

규모 설정은 준비돼 있어, 트래픽이 늘면 환경 준비와 데이터 전환 검증을 거쳐 적용합니다.

캐시 계층 활성화

TLS Redis는 이미 연결돼 있어 어댑터 구현과 함께 켭니다.